Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Шапошник А.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 19.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Отсортировать массив целых чисел при помощи параллельного алгоритма быстрой сортировки

Требования:

1)Использовать стандартные средства создания потоков ОС (POSIX Threads).

2)Реализовать последовательную и параллельную версии.

3)Ограничение максимального количества потоков задавать ключом запуска.

4)Провести исследование зависимости ускорения от количества потоков и размера массива.

Общий метод и алгоритм решения

Описание работы программы:

Программа реализует алгоритм быстрой сортировки (quicksort) в двух вариантах:

1. Последовательная версия: Классический рекурсивный алгоритм с выбором медианы трех в качестве опорного элемента и сортировкой вставками для подмассивов размером менее 16 элементов.

2. Параллельная версия: Использует модель Fork-Join. После выполнения операции разделения массив делится на две независимые части. Одна часть отправляется в новый поток, а другая обрабатывается в текущем потоке. Для предотвращения создания избыточного количества потоков используется самодельный семафор (на базе мьютекса и счетчика), который ограничивает максимальное число одновременно работающих потоков.

Код программы

consistently.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static void fill_random(int *arr, size_t n, unsigned seed){
    srand(seed);
    for (size_t i = 0; i < n; ++i)
        arr[i] = rand();
}

static int is_sorted(const int *arr, size_t n){
    for (size_t i = 1; i < n; ++i)
        if (arr[i - 1] > arr[i])
            return 0;
    return 1;
```

```c
}

static double check_time(void){
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
}


static void swap(int *a, int *b){
    int t = *a;
    *a = *b;
    *b = t;
}

static int partitioning(int *arr, int left, int right){
    int mid = left + (right - left) / 2;
    if (arr[left] > arr[mid]){
        swap(&arr[left], &arr[mid]);
    }
    if (arr[left] > arr[right]){
        swap(&arr[left], &arr[right]);
    }
    if (arr[mid] > arr[right]){
        swap(&arr[mid], &arr[right]);
    }

    int pivot = arr[mid];
    swap(&arr[mid], &arr[right - 1]);
    int i = left, j = right - 1;
    for (;;) {
        while (arr[++i] < pivot) {}
        while (arr[--j] > pivot) {}
        if (i >= j) break;
        swap(&arr[i], &arr[j]);
    }
    swap(&arr[i], &arr[right - 1]);
    return i;
}

static void quicksort(int *arr, int left, int right){
    while (left < right) {
        if (right - left < 16) {
            for (int i = left + 1; i <= right; ++i) {
                int key = arr[i], j = i - 1;
                while (j >= left && arr[j] > key) { arr[j + 1] = arr[j]; --j; }
                arr[j + 1] = key;
            }
            return;
        }
        int p = partitioning(arr, left, right);
        if (p - left < right - p) {
            quicksort(arr, left, p - 1);
            left = p + 1;
        } else {
            quicksort(arr, p + 1, right);
            right = p - 1;
```

```
        }
    }
}

int main(int argc, char **argv){
    size_t n = 5000000;
    if (argc > 1) n = strtoull(argv[1], NULL, 10);

    int *arr = malloc(n * sizeof(int));
    if (arr == NULL) {
        perror("malloc");
        return 1;
    }
    fill_random(arr, n, 13);

    double t0 = check_time();
    quicksort(arr, 0, (int)n - 1);
    double t1 = check_time();

    printf("[последовательная версия] n=%zu time=%.3f ms sorted=%s\n", n, t1 - t0,
is_sorted(arr, n) ? "yes" : "NO");
    free(arr);
    return 0;
}
```

parallel.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <getopt.h>
#include <pthread.h>

void fill_random(int *arr, size_t n, unsigned seed){
    srand(seed);
    for (size_t i = 0; i < n; ++i){
        arr[i] = rand();
    }
}

int is_sorted(const int *arr, size_t n){
    for (size_t i = 1; i < n; ++i)
        if (arr[i - 1] > arr[i]) return 0;
    return 1;
}

double check_time(void){
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
}

typedef struct {
    int value;
    pthread_mutex_t mtx;
} MySem;
```

```c
void mysem_init(MySem *s, int initial_value){
    s->value = initial_value;
    pthread_mutex_init(&s->mtx, NULL);
}

void mysem_destroy(MySem *s){
    pthread_mutex_destroy(&s->mtx);
}

int mysem_trywait(MySem *s){
    int ret = -1;
    pthread_mutex_lock(&s->mtx);
    if (s->value > 0) {
        s->value--;
        ret = 0;
    }
    pthread_mutex_unlock(&s->mtx);
    return ret;
}

void mysem_post(MySem *s){
    pthread_mutex_lock(&s->mtx);
    s->value++;
    pthread_mutex_unlock(&s->mtx);
}

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int *arr, int left, int right){
    int mid = left + (right - left) / 2;
    if (arr[left] > arr[mid])   swap(&arr[left], &arr[mid]);
    if (arr[left] > arr[right]) swap(&arr[left], &arr[right]);
    if (arr[mid] > arr[right])  swap(&arr[mid], &arr[right]);
    int pivot = arr[mid];
    swap(&arr[mid], &arr[right - 1]);
    int i = left, j = right - 1;
    for (;;) {
        while (arr[++i] < pivot) {}
        while (arr[--j] > pivot) {}
        if (i >= j) break;
        swap(&arr[i], &arr[j]);
    }
    swap(&arr[i], &arr[right - 1]);
    return i;
}

void quicksort(int *arr, int left, int right){
    while (left < right) {
        if (right - left < 16) {
            for (int i = left + 1; i <= right; ++i) {
                int key = arr[i], j = i - 1;
                while (j >= left && arr[j] > key){
                    arr[j + 1] = arr[j]; --j;
```

```
                }
                arr[j + 1] = key;
            }
            return;
        }
        int p = partition(arr, left, right);
        if (p - left < right - p){
            quicksort(arr, left, p - 1);
            left = p + 1;
        }else{
            quicksort(arr, p + 1, right);
            right = p - 1;
        }
    }
}

typedef struct {
    int    *arr;
    int     left;
    int     right;
    MySem  *sem;
} Inner;

void *qs_run(void *arg){
    Inner *in = (Inner *)arg;
    int *arr   = in->arr;
    int  left  = in->left;
    int  right = in->right;
    MySem *sem = in->sem;

    while (left < right) {
        if (right - left < 16) {
            quicksort(arr, left, right);
            break;
        }

        int p = partition(arr, left, right);
        int spawned = 0;

        if ((right - p) >= 16 && mysem_trywait(sem) == 0) {
            Inner *r = malloc(sizeof(Inner));
            if (r) {
                *r = (Inner){arr, p + 1, right, sem};
                pthread_t tid;
                if (pthread_create(&tid, NULL, qs_run, r) == 0) {
                    spawned = 1;
                    Inner l = {arr, left, p - 1, sem};
                    qs_run(&l);
                    pthread_join(tid, NULL);
                    free(r);
                } else {
                    free(r);
                    mysem_post(sem);
                }
            } else {
                mysem_post(sem);
            }
```

```
        }

        if (spawned) {
            mysem_post(sem);
            return NULL;
        }
        quicksort(arr, left, p - 1);
        left = p + 1;
    }
    return NULL;
}

int main(int argc, char **argv){
    size_t n = 5000000;
    int max_threads = 4;
    int opt;

    while ((opt = getopt(argc, argv, "n:t:")) != -1) {
        switch (opt) {
        case 'n': n = strtoull(optarg, NULL, 10); break;
        case 't': max_threads = atoi(optarg);     break;
        }
    }

    if (max_threads < 1) {
        fprintf(stderr, "error: -t must be >= 1\n");
        return 1;
    }

    int *arr = malloc(n * sizeof(int));
    if (!arr) { perror("malloc"); return 1; }
    fill_random(arr, n, 13);

    MySem sem;
    mysem_init(&sem, max_threads);

    Inner root = { arr, 0, (int)n - 1, &sem };

    double t0 = check_time();
    qs_run(&root);
    double t1 = check_time();

    mysem_destroy(&sem);

    printf("[параллельная версия с числом потоков t=%d] n=%zu time=%.3f ms sorted=%s\n",
           max_threads, n, t1 - t0, is_sorted(arr, n) ? "yes" : "NO");

    free(arr);
    return 0;
}
```

Протокол работы программы

Далее продемонстрируем таблицу с зависимостью ускорения и эффективности от количества потоков, максимально работающих в один момент. Ускорение будем рассчитывать по формуле $S = T_S / T_P$, где S-ускорение ($1 <= S <= P$), T_S- время последовательной реализации, T_P- время параллельной реализации, P-количество ядер. Эффективность рассчитываем по формуле $X = S / P$, где X- Эффективность ($X < 1$). Работа выполняется на 4-ядерном процессоре Intel Core i5, таким образом количество логических ядер составляет 8.

Число потоков	Время(мс)	Ускорение	Эффективность
1 (последовательная реализация)	847.331	1	1
2	445.331	1,9	0,95
4	431.060	1,97	0,4925
6	403.412	2,1	0,35
8(количество логических ядер)	367.823	2,3	0,2875
16	254.384	3,3	0,20625
128	363.655	2,33	0,018
1024	640.387	1,3	0,0013

```
[последовательная версия] n=10000000 time=847.331 ms sorted=yes
```

```
Artem@MacBookPro Second laba % ./parallel -n 10000000 -t 2
[параллельная версия с числом потоков t=2] n=10000000 time=445.331 ms sorted=yes
Artem@MacBookPro Second laba % ./parallel -n 10000000 -t 4
[параллельная версия с числом потоков t=4] n=10000000 time=431.060 ms sorted=yes
Artem@MacBookPro Second laba % ./parallel -n 10000000 -t 6
[параллельная версия с числом потоков t=6] n=10000000 time=403.412 ms sorted=yes
Artem@MacBookPro Second laba % ./parallel -n 10000000 -t 8
[параллельная версия с числом потоков t=8] n=10000000 time=367.823 ms sorted=yes
Artem@MacBookPro Second laba % ./parallel -n 10000000 -t 16
[параллельная версия с числом потоков t=16] n=10000000 time=254.384 ms sorted=yes
Artem@MacBookPro Second laba % ./parallel -n 10000000 -t 128
[параллельная версия с числом потоков t=128] n=10000000 time=363.655 ms sorted=yes
Artem@MacBookPro Second laba % ./parallel -n 10000000 -t 1024
[параллельная версия с числом потоков t=1024] n=10000000 time=640.387 ms sorted=yes
```

Вывод

В ходе выполнения лабораторной работы была успешно реализована параллельная версия алгоритма быстрой сортировки (Quicksort) с использованием стандарта POSIX Threads и модели Fork-Join. Для ограничения максимального количества одновременно работающих потоков был разработан собственный семафор на базе мьютекса и счетчика, а для защиты от избыточных накладных расходов введен порог гранулярности задач.